

Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl



¿Por qué limpiar los datos?

- Los datos del mundo real son "**sucios**" y contienen errores o inconsistencias.
- Si los datos de entrada tienen fallas, los modelos predictivos generarán predicciones erróneas.
- *Principio GIGO*: "Garbage In, Garbage Out" (Si entra basura, sale basura).
- En Python, automatizamos este proceso usando librerías como Pandas y Scikit-Learn.

Limpieza de Datos: El Problema de la Realidad

En el sector financiero, los datos nunca llegan perfectos.

- **Valores Nulos (NaN):**

- *Causa:* Error de sistema, campo no obligatorio, cliente no reportado.
- *Riesgo:* Sesgo en modelos de scoring.

- **Duplicados:**

- *Causa:* Re-procesamiento de archivos batch, doble entrada manual.
- *Riesgo:* Inflar la exposición al riesgo o contar doble una deuda.

Definición del Caso Práctico

Objetivo: Generar un reporte de calidad de cartera.

- **Dataset:** Datos simulados de 10,000 créditos.
- **Variables:** `id_cliente`, `monto`, `dias_mora`, `score_buro`.
- **Reglas de Negocio:**
 - **Cliente Cumplido:** Días de mora < 30 .
 - **Cliente Moroso:** Días de mora ≥ 90 .
 - **En Observación:** Días de mora entre 30 y 89.

Filtrado de Clientes (Boolean Indexing)

Usamos máscaras booleanas para segmentar la cartera.

```
# Filtro de Morosos
filtro_mora = df['dias_mora'] >= 90
morosos = df[filtro_mora]

# Filtro de Cumplidos
filtro_cumplidos = df['dias_mora'] < 30
cumplidos = df[filtro_cumplidos]

# Cálculo de tasa de incumplimiento
tasa_mora = len(morosos) / len(df) * 100
```

Reporte de Calidad de Datos

Antes de modelar riesgo, debemos reportar la salud de la base.

- **Integridad:** % de Nulos por columna.
- **Unicidad:** % de IDs duplicados.

```
def reporte_calidad(df):  
    nulos = df.isnull().mean() * 100  
    duplicados = df.duplicated().sum()  
    print(f"Duplicados totales: {duplicados}")  
    print("Porcentaje de nulos:\n", nulos)
```

Sección 1: Valores Faltantes

¿Qué hacer cuando faltan datos en Pandas?

Estrategias para tratar datos faltantes en crédito:

1. Eliminación

- Útil si la fila carece de ID o Target.
- `df.dropna()`

2. Imputación

- Llenar con media/mediana (ingresos).
- Llenar con "Desconocido" (categóricas).
- `df.fillna(value)`

```
# Ejemplo: Imputar ingresos con la mediana
mediana_ingreso = df['ingreso'].median()
df['ingreso'] = df['ingreso'].fillna(mediana_ingreso)
```

Manejo de Duplicados

Identificar y eliminar registros repetidos es crucial para el reporte regulatorio.

```
# Verificar duplicados totales
duplicados = df.duplicated().sum()

# Eliminar duplicados basandonos en ID de crédito
# keep='last' mantiene el registro más reciente (actualización)
df_clean = df.drop_duplicates(subset=['id_credito'], keep='last')
```

Manejo de data heterogénea

En el análisis univariado, nos enfocamos en una sola variable. Para detectar datos ruidosos, se definen intervalos de confianza donde la mayoría de los datos "normales" deberían caer. Observaciones fuera de estos intervalos se consideran potencialmente ruidosas y podrían requerir limpieza, como eliminación, imputación o investigación adicional.

- Distribución Similar a Normal: Para distribuciones que se aproximan a una normal (gaussiana), el intervalo contiene aproximadamente el 96 % de los datos, donde μ es la media y σ es la desviación estándar.

$$[\mu - 2\sigma, \mu + 2\sigma]$$

Manejo de data heterogénea

En el manejo de datos ruidosos, este intervalo se usa para identificar outliers: por ejemplo, en un conjunto de datos de temperaturas, valores fuera de este rango podrían indicar sensores defectuosos. Una regla común es el z-score: si $|z| > 2$ (donde $z = (x - \mu)/\sigma$), el punto es sospechoso de ser ruido. Para un umbral más estricto, se usa 3σ (99.7 % de los datos), pero 2σ es útil para detección inicial en datasets con ruido moderado.

En el caso general (sin asumir normalidad), el teorema de Chebyshev establece que, para cualquier $\gamma > 1$, el intervalo

$$[\mu - \gamma\sigma, \mu + \gamma\sigma]$$

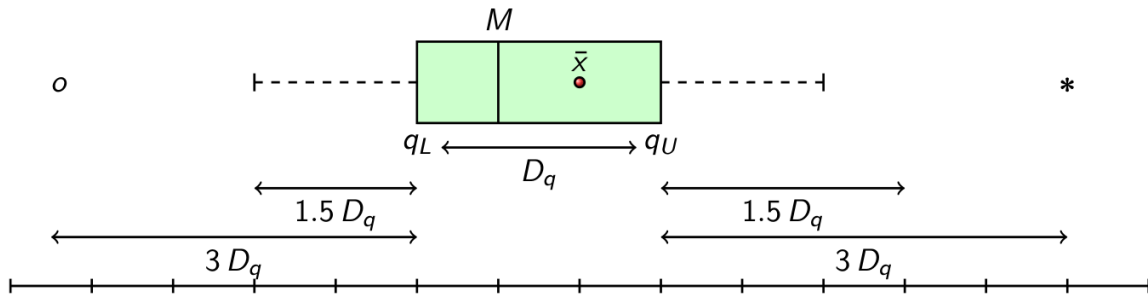
contiene al menos la proporción $1 - 1/\gamma^2$ de las observaciones.

Manejo de data heterogénea

Este teorema es conservador y aplica a cualquier distribución, lo que lo hace ideal para datos ruidosos con distribuciones desconocidas o sesgadas. Por ejemplo, si $\gamma = 2$, al menos el 75 % de los datos están dentro del intervalo (aunque en normales es 95 %). En práctica, para manejar ruido:

- Elige γ basado en la tolerancia al ruido (e.g., $\gamma = 3$ garantiza al menos 89 % dentro).
- Marca observaciones fuera como ruidosas; útil en finanzas para detectar transacciones anómalas sin asumir normalidad.
- Ventaja: Robusto a distribuciones no normales, pero menos preciso que métodos paramétricos.

Box-plot



$$D_q = q_U - q_L = q_{0.75} - q_{0.25}$$

$$\text{internal lower edge} = q_L - 1.5 D_q$$

$$\text{external lower edge} = q_L - 3 D_q$$

¿Qué son los valores faltantes (NaN)?

- Representados en Python y NumPy como `np.nan` o `None`.
- **Causas comunes:**
 - Omisiones en formularios o encuestas.
 - Fallas en la recolección o ingesta automatizada.
 - Cruces de bases de datos fallidos (joins vacíos).

¡Ojo!

Funciones estadísticas en Pandas (como `.mean()`) ignoran por defecto los nulos, pero los algoritmos de Machine Learning fallarán si los detectan.

Detección de Nulos en Pandas

Para detectar dónde y cuántos datos faltan utilizamos los métodos `.isna()` y `.sum()`:

```
import pandas as pd
import numpy as np

# Cargar dataset y verificar nulos por columna
data = pd.read_csv("credito.csv")
print(data.isna().sum())
```

- `data.isna()`: Devuelve un DataFrame de booleanos (True si es nulo).
- `.sum()`: Cuenta la cantidad total de True acumulados por columna.

Tratamiento 1: Eliminación con dropna()

- **Método:** `data.dropna(axis=0)` elimina todas las filas que contengan al menos un valor nulo.
- **Cuándo usarlo:** Si la cantidad de nulos es mínima y no afectará drásticamente el tamaño total de la muestra.

```
# Eliminar filas con cualquier valor faltante
data_limpia = data.dropna()
print("Forma antes:", data.shape)
print("Forma despues:", data_limpia.shape)
```

Tratamiento 2: Imputación con `fillna()`

En lugar de descartar filas valiosas, rellenamos los espacios vacíos con estimaciones estadísticas:

```
# Rellenar con la media de la columna o una constante
data_rellena = data.copy()
data_rellena['cuota'] = data_rellena['cuota'].fillna(data_rellena['cuota'].mean())
data_rellena['ID'] = data_rellena['ID'].fillna(-1)
```

Buenas prácticas:

- Para variables numéricas: Usar la media o la mediana.
- Para identificadores o códigos: Usar un valor centinela único (ej. -1).

Gestión de Valores Duplicados en Pandas

Registros idénticos repetidos pueden sesgar el entrenamiento de modelos supervisados.

```
# Verificar cu ntos duplicados existen
duplicados = data[data.duplicated()]
print(f"Filas duplicadas: {len(duplicados)}")

# Eliminar duplicados conservando la primera ocurrencia
data.drop_duplicates(inplace=True)
```

- El método `.duplicated()` busca filas idénticas en todo el DataFrame.
- `drop_duplicates(inplace = True)` aplica los cambios directamente sobre la estructura de datos.

Sección 2: Estandarización de Datos

Alineando escalas para Machine Learning.

¿Por qué es necesaria la Estandarización?

- Los algoritmos basados en distancias (KNN, SVM, Redes Neuronales) o descensos de gradiente son sensibles a las escalas.
- **Ejemplo:** Si una variable mide ingresos (en miles) y otra mide tasas de interés (en porcentajes), la primera dominará numéricamente el cálculo de distancias.
- **Solución:** Reescalar las características para que todas posean propiedades estadísticas similares.

Estandarización Min-Max (Normalización)

- **Propósito:** Reescalar todas las características numéricas dentro de un rango acotado y fijo, comúnmente entre **0 y 1**.

- **Fórmula matemática:**

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Cuándo usarla:** En algoritmos que dependen de distancias acotadas (como redes neuronales o K-Nearest Neighbors) o cuando se requiere que no existan valores negativos.

```
from sklearn.preprocessing import
    MinMaxScaler

# Instanciar el escalador
scaler = MinMaxScaler(feature_range
    =(0, 1))

# Ajustar y transformar datos
datos_minmax = scaler.fit_transform(
    df_numerico)
```

Nota importante

A diferencia del *StandardScaler*, el método Min-Max es **muy sensible a los outliers**, ya que un valor extremo modifica drásticamente el valor de x_{max} o x_{min} .

Estandarización usando StandardScaler

El escalador estándar centra los datos restando la media y dividiendo entre la desviación estándar:

$$z = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
# Ajustar y transformar matriz num rica
datos_escalados = scaler.fit_transform(df_numerico)
df_scaled = pd.DataFrame(datos_escalados, columns=df_numerico.columns)
```

Tras este proceso, cada columna presentará una $\mu \approx 0$ y un $\sigma = 1$.

Sección 3: Detección de Outliers

Identificando observaciones anómalas.

¿Qué es un Outlier?

- Es una observación numérica que se desvía drásticamente del comportamiento general del resto de la muestra.
- Pueden surgir por errores de medición, transgresiones físicas reales o eventos extraordinarios.
- En modelos de regresión lineal, un solo outlier severo puede alterar significativamente la pendiente de la línea ajustada.

Detección Gráfica mediante Boxplots

Visualizamos los valores extremos de forma univariada usando Seaborn:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6, 3))
sns.boxplot(x=df_numerico['ingreso'], color='orange')
plt.title('Boxplot de Ingresos')
plt.show()
```

Interpretación del diagrama de caja: Los puntos aislados ubicados fuera de los bigotes superior e inferior representan estadísticamente los outliers.

Filtrado Estadístico con Z-Score

Podemos eliminar outliers de manera algorítmica utilizando la regla empírica de las 3 desviaciones estándar:

```
columna = 'ingreso'
media = df_numerico[columna].mean()
desv = df_numerico[columna].std()

# Conservar solo valores dentro de +- 3 desviaciones
df_sin_outliers = df_numerico[
    np.abs(df_numerico[columna] - media) <= 3 * desv
]
```

Cualquier registro cuyo valor absoluto de desviación supere el umbral de 3 es descartado del análisis.

Detección Multivariada: Isolation Forest

Cuando analizamos múltiples dimensiones a la vez, los métodos univariados fallan. Usamos aprendizaje no supervisado:

```
from sklearn.ensemble import IsolationForest

# Configurar el modelo de aislamiento de anomalías
clf = IsolationForest(contamination=0.05, random_state=42)
predicciones = clf.fit_predict(df_numerico)

# Las etiquetas devuelven -1 para outliers y 1 para normales
df_filtrado = df_numerico[predicciones == 1]
```

- **contamination:** Porcentaje estimado de valores atípicos en el dataset.

Sección 4: Exploración de Datos

Análisis Univariado y Bivariado.

En Pandas, el análisis exploratorio se apoya fuertemente en resúmenes tabulares:

```
# Estadísticos descriptivos globales (media, desviación, cuartiles)
display(df_numerico.describe())

# Agregaciones segmentadas por perfil crediticio
display(df_numerico.groupby('perfil').mean())
```

Permite contrastar rápidamente cómo se comportan las métricas financieras (como cuotas o deudas) dependiendo de la categoría de perfil del cliente.

Visualización Bivariada Avanzada (pairplot)

Para examinar correlaciones cruzadas entre múltiples variables simultáneamente:

```
variables = ['oferta', 'cuota', 'ingreso', 'perfil']  
sns.pairplot(df_numerico[variables], hue='perfil', palette='tab10')  
plt.show()
```

- Genera una matriz de gráficos de dispersión (scatter plots) para relaciones bivariadas.
- La diagonal muestra la distribución univariada (KDE o histogramas) discriminada por color según el perfil.

El Pipeline Profesional de Preprocesamiento

Un flujo de trabajo robusto en ciencia de datos integra todos los pasos anteriores de forma secuencial:

- ➊ **Limpieza estructural:** Carga del archivo CSV[cite: 1] y eliminación de filas duplicadas.
- ➋ **Imputación de nulos:** Reemplazo estratégico mediante `fillna()` para evitar pérdida de registros.
- ➌ **Control de extremos:** Depuración de outliers univariados (Z-score) o multivariados (Isolation Forest).
- ➍ **Transformación numérica:** Escalado con `StandardScaler` previo al modelado predictivo.

Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl

